Øving 5

The algorithm

Edmonds-Karp finds the maximum flow in a weighted directed graph by repeatedly searching for *augmenting paths* - paths from the source to the sink along which more flow can still be pushed. Its defining choice is how those paths are found: by breadth-first search (BFS). This makes every augmenting path a *shortest* path in number of edges, which is what distinguishes Edmonds-Karp from the generic Ford-Fulkerson method.

The search operates on the residual network, not the original graph. Every edge keeps a residual capacity (capacity minus flow already committed), and BFS may only traverse edges with residual capacity > 0. Two details make the residual network work:

1. **Back-edges.** Every edge exists in both directions. Pushing f units of flow along an edge $u \to v$ consumes f of its residual capacity *and creates* f units of residual capacity on the reverse edge $v \to u$. Intuitively, flow already sent can be "undone" or rerouted later.
2. **Bottleneck.** Once BFS finds the sink, the predecessor chain gives one augmenting path. The path can only carry as much as its tightest edge allows, so the flow increase equals the minimum residual capacity along the path. Every edge on the path is then updated: $+f$ forward, $-f$ backward.

The algorithm terminates when BFS can no longer reach the sink. At that point the flow is maximum - guaranteed by the max-flow/min-cut theorem: the set of nodes BFS *can* reach forms the source side of a minimum cut, and every edge crossing it is fully saturated.

The BFS choice also gives a complexity guarantee: each edge becomes critical at most $O(V)$ times, so there are at most $O(VE)$ augmenting paths, each found in $O(E)$ by BFS - $O(VE^2)$ overall, independent of the capacity magnitudes.

Results

Graph	Source	Sink	Augmenting paths	Maximum flow
flytgraf1	0	7	8	**10** (matches assignment)
flytgraf2	0	1	4	**27** (matches assignment)

Graph	Source	Sink	Augmenting paths	Maximum flow
flytgraf3	0	1	4	**42**
flytgraf4	0	7	8	**11** (matches assignment)
flytgraf5	0	7	5	**90**

Full program output (`increase : augmenting path`):

```
Maksimum flyt fra 0 til 7 med Edmonds-Karp (flytgraf1)
2 : 0 4 3 7
1 : 0 1 2 3 7
1 : 0 4 5 6 7
1 : 0 1 2 3 6 7
1 : 0 1 4 5 6 7
1 : 0 1 2 3 5 6 7
1 : 0 1 2 4 5 6 7
2 : 0 1 2 3 4 5 6 7
Maksimal flyt ble 10

Maksimum flyt fra 0 til 1 med Edmonds-Karp (flytgraf2)
10 : 0 8 9 1
10 : 0 2 3 9 8 14 15 1
5 : 0 2 3 4 5 8 9 12 13 14 15 1
2 : 0 2 3 4 5 6 7 9 8 10 11 12 13 14 15 1
Maksimal flyt ble 27

Maksimum flyt fra 0 til 1 med Edmonds-Karp (flytgraf3)
10 : 0 8 9 1
15 : 0 2 3 9 8 14 15 1
11 : 0 2 3 4 5 8 9 12 13 14 15 1
6 : 0 2 3 4 5 6 7 9 8 10 11 12 13 14 15 1
Maksimal flyt ble 42

Maksimum flyt fra 0 til 7 med Edmonds-Karp (flytgraf4)
2 : 0 4 3 7
1 : 0 1 2 3 7
1 : 0 4 5 6 7
1 : 0 1 2 3 6 7
1 : 0 1 4 5 6 7
1 : 0 1 2 3 5 6 7
1 : 0 1 2 4 5 6 7
3 : 0 1 2 3 4 5 6 7
Maksimal flyt ble 11

Maksimum flyt fra 0 til 7 med Edmonds-Karp (flytgraf5)
10 : 0 2 3 7
25 : 0 2 4 7
10 : 0 5 6 7
40 : 0 5 6 1 7
```

```
5 : 0 2 4 6 1 7
Maksimal flyt ble 90
```

flytgraf3 - maximum flow 42

Four augmenting paths suffice, and the run is a clean demonstration of back-edge exploitation. The graph has no edge $9 \to 8$ in the file - only $8 \to 9$ with capacity 10. Yet the second path is $0\ 2\ 3\ 9\ 8\ 14\ 15\ 1$, traversing $9 \to 8$. That is only possible because the first path pushed 10 units through $8 \to 9$, creating 10 units of residual capacity in the reverse direction. The algorithm is literally rerouting flow that it placed one step earlier - something a plain DFS on the original graph could never do.

Correctness is confirmed by exhibiting a cut of the same value. Take $S = \{0, 2, 3, 4, 5, 6, 7\}$ (everything reachable from the source on the "upper" branch) against $T = \{8, 9, 10, 11, 12, 13, 14, 15, 1\}$. The crossing edges are:

$$0 \to 8\ (10) + 7 \to 9\ (6) + 5 \to 8\ (11) + 3 \to 9\ (15) = 42$$

Since any flow is bounded by any cut, no flow greater than 42 can exist - and the algorithm achieved 42, so it is maximal.

flytgraf5 - maximum flow 90

This graph has a notable structural feature: node 1 is *not* the sink but sits on the path to it, and the edge $1 \to 7$ with capacity 50 carries the single largest contribution to the flow. The largest single augmentation (40 units along $0\ 5\ 6\ 1\ 7$) uses it.

The minimum cut here is small and elegant: $S = \{0, 2, 4, 3\}$ against $T = \{5, 6, 1, 7\}$, with crossing edges

$$0 \to 5\ (50) + 4 \to 6\ (5) + 3 \to 7\ (10) + 4 \to 7\ (25) = 90$$

Every one of these four edges ends up saturated, which is exactly why BFS can no longer find an augmenting path when the algorithm stops. Note that the edge $5 \to 4$ (capacity 20) points *into* S and therefore does not count toward the cut - a common exam trap. The flow of 90 is thus provably optimal.

Observations

- **Flow reversal.** In flytgraf1 and flytgraf4, the first path routes 2 units through $4 \to 3$. The final path ($0\ 1\ 2\ 3\ 4\ 5\ 6\ 7$) pushes flow through $3 \to 4$, cancelling it - the net flow on $4 \to 3$ ends at 0 (flytgraf1) or is partially reversed (flytgraf4, where the last augmentation is 3 units). This is the behaviour the assignment text highlights, and it falls out naturally from the residual back-edges.
- **Path counts differ, flow values don't.** The professor's example run for flytgraf1 found the same 8 paths in a different order; our run for flytgraf2/3 uses 4 paths where other orderings might use more. Edmonds-Karp fixes *which* shortest paths are chosen only up to BFS tie-breaking, but the max-flow/min-cut theorem guarantees the final flow value is unique.
- **Few paths, large increments.** The bigger graphs need remarkably few augmentations (4-5) because BFS finds short paths with large bottlenecks first - consistent with the $O(VE)$ path bound being a worst case, not a typical case.

Embedded flow-network files

The five graph files are compiled into the executable using the C++26 `#embed` directive, which expands a file's bytes directly into a `constexpr` array - no runtime file I/O, and the binary is fully self-contained. This requires GCC 15 or newer with `-std=c++26` : GCC 15 introduced `#embed` for C++26 (P1967R14), and the compile-time error handling (throwing from `constexpr` functions) works on GCC 15.2 as verified on the school's Linux shells, though it is formally specified as a GCC 16 feature.